NETBOX PLUGIN

User PIN – complete guide

A plugin for NetBox 4.7+

Version 2.0.1 · 2026-10-31
github.com/ArK761/NetBox_UserPIN

Contents

1. About the plugin
2. Who is who
3. Installation
   - 3.1 Requirements
   - 3.2 Steps
   - 3.3 First start
   - 3.4 One-line plugin update
   - 3.5 Optional settings in the configuration
   - 3.6 Upgrading NetBox itself
4. My PIN, 2FA and unlock
   - 4.1 PIN
   - 4.2 Two-factor authentication (2FA)
   - 4.3 Backup codes
   - 4.4 Confirming administrative actions (step-up)
   - 4.5 Forgotten PIN (self-service)
   - 4.6 Emergency 2FA reset on the server
5. Settings and Mail
   - 5.1 Settings (User PIN → Settings)
   - 5.2 Mail
6. Roles: CORE and departments
   - 6.1 Resets without a meeting
   - 6.2 Four-eyes approval
7. Protected areas
   - 7.1 Groups (the way access is given)
   - 7.2 Field rules
   - 7.3 Tabs of the detail page
   - 7.4 Access requests
   - 7.5 What is enforced
   - 7.6 An area registered by a plugin
8. Global protection (ON / OFF)
9. Exceptions
   - 9.1 What a user with an exception may use (since 2.0.1)
10. Section managers
11. PIN-only accesses
   - 11.1 Signing in
   - 11.2 Limits of the person
   - 11.3 Wrong PINs (default 5 per step, waits 10, 20, 30 s; own rules on the PIN-only page, also per access)
   - 11.4 Managing an access
   - 11.5 E-mails both ways
12. Working time
13. E-mails at a glance
14. Audit log
15. Server commands
   - 15.1 Key rotation
16. Check: pin_check.sh
17. Security model
18. Using it from another plugin
19. Troubleshooting
20. Changes
   - 20.1 2.0.1 – permissions of exceptions
   - 20.2 Keys and security
   - 20.3 The new model of protection
   - 20.4 Section managers and requests
   - 20.5 PIN-only accesses
   - 20.6 Working time (since 2.2)
   - 20.7 Tools and upgrade preparation
   - 20.8 What the plugin does not do yet

1. About the plugin

User PIN is a plugin for NetBox 4.7+ (Python 3.12+). It gives every user a **personal PIN** and builds on it a protection of the sensitive parts of NetBox: protected areas, groups with rights, field rules, PIN-only accesses for people without a NetBox account, working time and a detailed audit.

- One PIN per user, valid for every plugin that uses User PIN.
- The PIN is stored as an Argon2id hash encrypted with AES-256-GCM. The key lives only in the NetBox configuration, not in the database.
- Two-factor authentication (2FA, TOTP: Microsoft / Google Authenticator …) with backup codes.
- Unlock for a set time, lockout after too many wrong attempts, weak PINs rejected.
- Roles **CORE** and **departments** (invitations, hand-over, moves), four-eyes approval.
- **Protected areas**: NetBox objects (an app or an object type) are seen only by those who hold access from the area, superusers included.
- **Global protection** "deny everything not allowed" with ON/OFF, **protection keepers**, **section managers** and **exceptions**.
- **PIN-only accesses** for people without an account, with working time and public holidays.
- E-mails both ways, an audit of everything, a check script and emergency server commands.
- Languages: English and Slovak (Settings → General) for the plugin pages and e-mails.

> **Names in the menu.** The interface uses: *My PIN*, *Protected areas*, *PIN-only access › People with a PIN only*, *Departments*, *Approvals*, *Audit log*, *Settings*, *Mail*.

2. Who is who

Who	Ruled by	Can do
PIN-only access (`PinUserNNNN`)	only its groups (always)	no API, no requests. Without a group it can only change its PIN. It is not a NetBox user (hidden from every user list).
Regular user in a group (protection ON)	only what the groups allow	no PIN prompt, no API key.
Regular user with an exception	NetBox permissions alone; of the account pages and the API only what a keeper allowed (the password by default)	PIN plugin pages by their role.
Section manager	NetBox as any user; **full power over his section**	groups, fields, members and requests of his section (an object type or a whole area); creates and runs the PIN-only accesses put under him.
Protection keeper	everything	areas, object types, section managers, exceptions, the ON/OFF switch, all PIN-only accesses; moves an access under a manager.
Master (superuser)	NetBox	invites CORE deputies; while there is no keeper he manages everything; while there is no CORE deputy he manages the PINs.

CORE deputy	—	everything except CORE; creates departments and appoints their heads.
Department head / **delegate**	—	the PINs of the department's members; the head invites delegates (at least 2 per department).

A keeper must have a PIN and 2FA. A section manager needs no department role, but needs a PIN and 2FA.

3. Installation

3.1 Requirements

NetBox 4.7.x, Python 3.12+. Dependencies of the plugin: `argon2-cffi`, `cryptography`, `segno`, `holidays` (public holidays; works offline).

3.2 Steps

```
source /opt/netbox/venv/bin/activate
pip install "argon2-cffi>=23.1" "cryptography>=42" "segno>=1.5" "holidays>=0.50"
pip install --upgrade --force-reinstall --no-deps "git+https://github.com/ArK761/NetBox_UserPIN.git
@2.0.0"
```

> **Mind `--no-deps`.** This option installs no dependencies. On a server where the plugin has never been, install them separately (the first line). Otherwise NetBox reports `ModuleNotFoundError: No module named 'argon2'`.

Generate the encryption key (the plugin is not running yet, use plain Python):

```
python3 -c "import base64,os; print(base64.urlsafe_b64encode(os.urandom(32)).decode())"
# or any long random text (at least 32 characters):
openssl rand -base64 64 | tr -d '\n'
```

A longer key is not stronger: the encryption always uses 256 bits. What matters is that it is **random**. Once the plugin runs, `python manage.py userpin_generate_key [--key-id k1]` does the same (it only prints, stores nothing).

Into `/opt/netbox/netbox/netbox/configuration.py`:

```
PLUGINS = ['netbox_user_pin']

PLUGINS_CONFIG = {
    'netbox_user_pin': {
        'encryption_keys': {'k1': '<generated key>'},
        'active_key_id': 'k1',
    },
}
```

> **Keep an offline backup of the key** and never put it into the same backup as the database. If it is lost, NetBox data stays intact, but everything the plugin stored under the key is gone: PINs, 2FA, backup codes, the SMTP password and the sign-in of the PIN-only people. Everybody sets them up again.

Migrations and restart:

```
cd /opt/netbox/netbox
python manage.py migrate netbox_user_pin
sudo systemctl restart netbox netbox-rq
```

3.3 First start

1. Sign in as superuser → **User PIN → My PIN** → set your PIN and **2FA** (needed for every administrative action).
2. In **Settings** and on the **Mail** page set the allowed e-mail domains and SMTP (see below).
3. On the **Users** page allow those who are to use a PIN.
4. In **Protected areas** appoint the first **protection keeper** (this switches the global protection on).

3.4 One-line plugin update

```
sudo bash -c 'source /opt/netbox/venv/bin/activate && pip install --upgrade holidays && pip install
--upgrade --force-reinstall --no-deps "git+https://github.com/ArK761/NetBox_UserPIN.git@<tag-or-com
mit>" && cd /opt/netbox/netbox && python manage.py migrate netbox_user_pin && systemctl restart net
box netbox-rq'
```

> **Production:** back up the database before every migration, for example `sudo -u postgres pg_dump netbox | gzip > /root/netbox-$(date +%F-%H%M).sql.gz`. Do not touch the keys in `configuration.py` meanwhile.

3.5 Optional settings in the configuration

Setting	Default	Meaning
`encryption_keys`	– (required)	`{key_id: key}`: urlsafe base64 32-byte key, or any secret of 32+ characters.
`active_key_id`	the only key	key used for new encryptions.
`argon2_time_cost`	3	Argon2id iterations.
`argon2_memory_cost`	65536	Argon2id memory in KiB (64 MiB).
`argon2_parallelism`	4	Argon2id lanes.
`cli_venv`	/opt/netbox/venv	shown in the emergency server command.
`cli_netbox_dir`	/opt/netbox/netbox	shown in the emergency server command.

Everything else is configured in the interface: **User PIN → Settings**.

3.6 Upgrading NetBox itself

`upgrade.sh` recreates the venv and installs only what is in `requirements.txt` and `local_requirements.txt`. A plugin that is not listed there disappears after the upgrade. Lines in `/opt/netbox/local_requirements.txt`:

```
netbox-librenms-plugin                # PyPI packages need just the name
netbox-qrcode
git+https://github.com/ArK761/NetBox_UserPIN.git@2.0.0
git+https://github.com/ArK761/NetboxLibre_OxI.git@<commit>
git+https://github.com/ArK761/NetboxDevice_Vulnerability.git@<commit>
git+https://github.com/ArK761/NetBoxSelf_Audit.git@<tag-or-commit>
```

- A line without `@…` installs the latest commit of the `main` branch, not the latest release. Pip does not know GitHub Releases, only branches, tags and commits.
- The code of some plugins lives on a branch other than `main`. Keep the pinned commit until you merge the branch.
- Upgrade only within **4.7.x**: the other plugins are limited to 4.7.0–4.7.99.
- After the upgrade check the plugin versions (the badge *User PIN vX · commit*) and run the check script (see the Check chapter).

4. My PIN, 2FA and unlock

4.1 PIN

- **Setting:** *My PIN* → set the PIN (6 digits by default; weak PINs such as 111111, 123456, 121212 and an own blocklist are rejected).
- **Unlock:** a protected page asks for the PIN ("enter your PIN to continue"). The unlock lasts the set time (15 minutes by default), can be extended on activity and applies either to everything or per area.
- **Lockout:** after 5 wrong attempts for 30 minutes. Wrong PINs and wrong 2FA codes are counted separately; either one reaching the limit locks.
- **Rotation:** after 180 days by default, with a warning 14 days before, a forced change and a button for a test rotation.
- A PIN change or reset invalidates every existing unlock of the user.

4.2 Two-factor authentication (2FA)

My PIN → 2FA: scan the QR code with an app (Microsoft / Google Authenticator …) and confirm with a code. A TOTP code is accepted only once (replay protection). 2FA is needed for every administrative action, for keepers, section managers and delegates.

4.3 Backup codes

My PIN → Show backup codes: after PIN + a 2FA code, or (without the phone) PIN + a verification code sent by e-mail. The codes are shown only in the browser (used ones crossed out) and new ones can be generated. The user gets an e-mail notification. The codes themselves are never e-mailed.

4.4 Confirming administrative actions (step-up)

An administrative action needs a fresh PIN + 2FA confirmation (valid a few minutes, 5 by default). It is confirmed in a pop-up directly on the page and the action continues without leaving it. Everyday actions (allow/deny a PIN, clear a lockout, force a PIN change, moves between departments) need only the PIN; resets, suspension, roles and dissolving a department need PIN + 2FA. *Require 2FA also for settings* extends this to the PIN-only actions. In the administration of areas the 2FA is asked only every set number of hours (8 by default).

4.5 Forgotten PIN (self-service)

1. *My PIN → Forgot PIN* (or the link on the unlock page) → **Send code**: an 8-digit code is e-mailed (only to an allowed domain, valid 15 minutes, max. 5 attempts, resend after 60 s).
2. Enter the e-mail code **and** a 2FA code (or a backup code) and the new PIN.

Without an e-mail in an allowed domain or without 2FA the recovery is not offered: an administrator resets the PIN.

4.6 Emergency 2FA reset on the server

```
sudo bash -c 'source /opt/netbox/venv/bin/activate && cd /opt/netbox/netbox && python manage.py use
rpin_reset_2fa <user>'
```

Shown at each account (*Users → Actions → Emergency: server command*) and in *My PIN*. The paths come from the settings `cli_venv` and `cli_netbox_dir`.

5. Settings and Mail

5.1 Settings (User PIN → Settings)

Option	Default	Meaning
Language	en	language of the plugin pages and e-mails (en / sk).
Show first and last name	on	next to the account in lists.
Who may use a PIN	only allowed users	only explicitly allowed users / everybody (superusers always).
PIN length	6	number of digits.
Block weak PINs; additional blocked PINs	on; empty	an own list of forbidden PINs.
Unlock duration (min); extend on activity; unlock scope	15; on; global	scope: one unlock for everything, or per area.
Require 2FA on every unlock	off	2FA also on an ordinary unlock.
Max failed attempts; lockout duration (min)	5; 30	wrong PINs and wrong 2FA codes are counted separately.
PIN max age (days); warn before expiry	180; 14	NIST SP 800-63B-4 does not recommend periodic changes: set according to your company policy.
Require PIN + 2FA for administrative actions; administrative window (min)	on; 5	validity of the confirmation.
Require 2FA also for settings	off	PIN-only actions will ask for 2FA too.

2FA asked again after (hours)	8	in the administration of areas.
Self-service recovery of a forgotten PIN; code validity (min)	on; 15	e-mailed code + 2FA.
Allowed e-mail domains	none	add and **verify** each domain (a code sent to an address in it).
Administrator reset valid (hours)	24	how long a reset waits for the user to confirm.
Four-eyes approval; for roles; request valid (min); break-glass	off; on; 1440; off	see the chapter Roles and approval.
Protect areas and objects	off	makes the areas defined on the Protected areas page protect.

5.2 Mail

E-mails (recovery codes, notifications) go through the SMTP server set on the **Mail** page: sender name (*NetBox PIN* by default), server, port (25 by default), encryption (off / STARTTLS / SSL), automatic TLS, SMTP authentication with a service account (the password is encrypted and never shown) and a connection test. If no server is set there, NetBox's own `EMAIL` setting is used.

- **Allowed domains** are written as `@company.com`, exact match only (no sub-domains or look-alikes). Verification: address → code by e-mail → enter the code. *Verify again* at any time.
- No e-mail goes to a domain before it is verified. Invitations can go only to users with an e-mail in a verified domain.
- Security notifications (changed, reset, locked, expiring) can be switched on/off; a test e-mail goes to a chosen user (the picker shows the account, the name and the address).

6. Roles: CORE and departments

```
CORE        master (superusers) + CORE deputies       everything
Department  head + delegates (min. 2) + members       the PINs of the department's members
```

Menu	Master	CORE	Head	Delegate	User
My PIN, My roles	✓	✓	✓	✓	✓
Users – PIN / 2FA / e-mail / expiry status	everybody	everybody	own department	own department	–
Users – allow/deny, suspend, resets, force change, clear lockout	all except self	all except master, CORE, self	members except self and other heads	members without a role	–
Departments	all	all	own: invite delegates, hand over, ask for moves	own: view	–
Audit log, Approvals	all	all	own department	own department	–
Settings, Mail	edit	view	–	–	–

- **Invitation:** every role (CORE deputy, head, delegate, temporary head, replacement) is an invitation: an e-mail with a link and a code, valid 12 / 24 / 36 / 48 h. The role starts only after the user accepts with the code, the PIN and 2FA.

- **Minimum:** with four-eyes at least 2 CORE deputies; every department a head and 2 delegates. A resignation below the minimum waits for a replacement. A head is removed only together with a new head.
- **Temporary hand-over:** the head (or CORE, or two delegates together for max. 30 days) hands the rights of the head to a member until a date. A reminder a day before; after the date the rights return automatically (system job *User PIN role housekeeping*, run every 5 minutes by `netbox-rq`).
- **Moves between departments:** a head asks, the head of the other department approves (CORE decides for users without a department and may move directly).
- Users can be filtered by department and role and shown as a tree (head → delegates → members). Colours: green = head + at least 2 delegates, yellow = delegates missing, red (blinking) = no head.
- Roles are synchronised into NetBox permissions named `User PIN: delegates …`, `User PIN: settings editors`, `User PIN: departments …`: do not edit them by hand.

6.1 Resets without a meeting

Action	Administrator	User (own session)
Suspend PIN (suspected leak, user absent)	blocks immediately, alone	–
Reset PIN	starts the reset (valid 24 h)	confirms with a **2FA code** and sets a new PIN
Reset 2FA	starts the reset	confirms with the **PIN** and enrolls the new phone

Until the user confirms, the old PIN keeps working; the administrator can cancel a pending reset. A user **without 2FA** cannot confirm anything: the administrator uses **Remove PIN** (PIN + 2FA of the administrator) and the user sets a new PIN the next time it is asked for.

6.2 Four-eyes approval

Settings → *Four-eyes approval* (needs at least two active CORE deputies with PIN and 2FA). Choose what it applies to: roles, settings (incl. mail), PIN access. Such a change then creates a request: the requester confirms with PIN + 2FA (and a reason), another eligible person gets a notification (bell) and an e-mail and confirms or rejects in *Approvals* with their own PIN + 2FA. The request page refreshes itself and both see the confirmations (time, IP). After the second confirmation the change is executed. Requests expire (24 h by default). **Break-glass** (off by default): a superuser may execute a request alone with a reason; all administrators are informed and it is marked in the audit log. Four-eyes cannot be switched off while a department exists; switching it off needs two people and ends the CORE delegation.

7. Protected areas

An area protects the objects of one or more apps (`dcim`, `ipam`, `netbox_libreoxi` …) or object types (`users.user`, `dcim.device` …), chosen from a list. Inside an area **nobody is a god**: the NetBox role and permissions of a user are not asked at all, the access (a group) given by the section managers or keepers decides, the master included. The door (menu entry, tab) stays visible for everybody; whoever has no key is refused on entering and told who manages the access.

7.1 Groups (the way access is given)

Under a protected object type (e.g. DCIM › Device) the manager creates **groups**: a named set of rights (e.g. *Reading*, *Writing*) with its tabs and fields, and puts people into them.

- Rights: **view**, **add**, **change**, **delete**, plus rights a plugin defines. Writing rights bring reading along.
- **Import and export** are rights of their own (*May import*, *May export*), off by default. Without them the Import/Export buttons are not drawn and the pages answer "no access". Importing needs *add* as well.
- **Records:** a group may apply only to the rows matching a condition (e.g. name contains `agg`) or to all.
- A user is in one group per type: adding him to another group moves him, leaving the last group means no access. An access can have a time or be unlimited.
- Several areas add up (union): whoever is granted nothing is denied.

7.2 Field rules

For a group and object type every field is **hidden**, **read only** or **read and write**. A field linked to other objects can be limited to the values matching a condition: *contains* (`~`), *starts with* (`^`), *is one of* (`=`, existing values); several values as chips; conditions are unfolded with the **Conditions** button. A rule without a group hides a field from everybody in the area. A hidden custom field keeps its stored data (saving an object never wipes it).

	custom fields	model fields
detail page	hidden / value filter	**not covered** (NetBox draws the fixed fields by hand)
edit forms	removed / disabled; the drop-down is narrowed and the value checked on save	removed / disabled
tables	column removed	column removed
REST API	omitted / value filter / writes ignored	omitted / read only

7.3 Tabs of the detail page

A tab stays in the tab bar (the door is visible); what it opens is decided like this: a **system tab** of NetBox opens only if it is ticked for that user in *Fields → Tabs of the detail page* (a new access allows the main page only); a tab that lists **other objects** (a device's interfaces) also needs the right on their type; the tab of a **plugin** follows the protected area of that plugin.

7.4 Access requests

On the "no access" page the user ticks the rights he needs (view / add / change / delete), writes a reason and sends it. An e-mail with a link goes to the **section manager and the keepers**. The link opens a page with who asks for what and why; the decider can change the rights and the duration, then **OK**. If there is no area for the object type, OK creates it and adds the user. Nobody decides about his own request. Every decision is e-mailed to the requester, the keepers and the manager. Requests can be switched off (*Users may ask for access*). Without a PIN the user is sent to create one (no 2FA needed).

7.5 What is enforced

Where	How
NetBox pages (lists, details, add/edit/delete, bulk actions, plugin pages)	middleware refuses without the right; the PIN has to be unlocked.
Rows	NetBox's `RestrictedQuerySet.restrict` is wrapped: in a protected area it returns the rows the group allows (none without one): lists, details, REST API, drop-downs.
REST API and GraphQL	refused without the right (403 JSON).
Menu and tabs	`has_perm` is answered by the PIN accesses.
Change log, journal, attachments, search, widgets	never show rows of objects a user may not view.

Not covered: filters in lists (`?cf_name=…`), exports, the fixed fields of the detail page and lists mixing several types. Anybody with root / database access on the server can bypass any plugin: the answer is the audit, not a technical lock.

7.6 An area registered by a plugin

A plugin can register an area (`areas.register(...)`); it is protected when its own "PIN integration" switch is on. Bootstrap: until an area has managers only the keepers (without any keeper: the superusers) have access, so nobody gets locked out. Sensitive apps (users, auth, core …) can be put into an area only by a keeper.

8. Global protection (ON / OFF)

On the **Protected areas** page, in the top card *Protection* (keepers only).

- **ON:** what no group allows is denied to everybody except keepers and exceptions. PIN-only accesses see only their groups. The denial reaches every path: lists, details, REST API, GraphQL, search, widgets, menus and buttons (a superuser who is no keeper sees none of Users, Groups, Permissions, Jobs, background tasks …). The own profile, bookmarks, notifications and preferences stay open, but only the user's own.
- **OFF:** everybody works by the NetBox permissions alone; PIN-only accesses can only change their PIN and are told that the function is switched off by the administrator.
- **The first keeper** is appointed by the master; appointing him switches the protection on. A keeper needs a PIN and 2FA, cannot turn 2FA off and cannot leave alone (another keeper has to take over); **the last keeper cannot be removed**. If the account of the last keeper is deleted the protection goes OFF and cannot be switched on before a keeper exists.
- **What is not allowed looks like:** *A – a door* (the menu is visible, opening says "no access" and offers a request) or *B – nothing* (the user sees nothing but the password and the pages of the plugin).
- **Users may ask for access** (on by default) and **may create their own API key** (off by default: a key works with the NetBox permissions and the PIN does not judge it, so allow it only for narrow permissions).
- **Whom the protection applies to:** *Only people with a PIN* (default: the PIN-only accesses and the users put into a group or made section managers; everybody else works by NetBox and does not see the plugin) or *Everybody*.

Emergency on the server (e-mails every keeper and superuser, audited):

```
python manage.py userpin_keeper --add <user>
python manage.py userpin_default_deny --on|--off
```

9. Exceptions

A user with an exception is not bound by the protection: he works with the permissions NetBox gives him. His groups, accesses and roles stay as they are and apply again when the exception is taken away or ends. Only a keeper sets it on the page *Protected areas → Exceptions*: the user, an optional end and a reason. The other keepers are told by e-mail and the audit log records it. A superuser with an exception sees everything without the PIN.

9.1 What a user with an exception may use (since 2.0.1)

An exception is no longer "all or nothing". With every exception the keeper ticks what the user may use of the account pages and the API:

Item	Default
Change the password (the Password tab)	allowed
Profile	not allowed
Preferences	not allowed
Bookmarks	not allowed
Notifications (the bell too)	not allowed
Subscriptions	not allowed
API tokens (create and manage their own)	not allowed
API (REST, GraphQL) with a token	not allowed

- Under the name of every user there is an arrow ▾ **May use** that unfolds the check boxes with a **Save** button, and small tags: green ✓ allowed, red ✗ not allowed.
- What is not allowed disappears from the menu and the tabs and the direct address returns **403**. A token of such a user is refused until the API is allowed.
- NetBox has no link to the password in the user menu (only a tab of the profile). While the password is allowed, the plugin adds one to the menu.
- A user who already has an exception is not offered in the list for adding; adding him again is refused and nothing is reset. Changes are made in the list.
- **A keeper can always do everything**, the limits do not apply to him. While the protection is off nothing is taken away. Existing exceptions get only the password after the migration.

> **Why it matters.** NetBox gives every signed-in user the right to their own API tokens (`DEFAULT_PERMISSIONS`). A user without any rights therefore sees *Admin → API Tokens* in the menu. A token carries only the permissions of its owner, but the plugin does not judge requests with a token, so a user with rights could read beyond his sections through a token. The list of allowances solves it without touching the NetBox configuration.

10. Section managers

- Appointed by a **keeper** with a section (the object types of the area; nothing ticked = the whole area). Needs a PIN and 2FA, no department role.
- The manager has **full power over his section**: groups, fields, members and requests. The keepers are above him.
- Requests for his types go to **him and the keepers**; every decision is e-mailed to the requester, the keepers and the manager.
- A PIN-only access made by a keeper belongs to the keepers. A manager sees only those put under him and creates them only into the groups of his section (the keepers are e-mailed). A keeper can move an access under a manager.
- Creating an area, targets, profiles and adding/removing managers belong to the keepers; a manager of a whole area keeps his rights over the area.
- Emergency: `python manage.py userpin_area_owners <area-key> --set name1 name2` (refuses people without a PIN and 2FA, superusers and keepers unless `--force` is given).

11. PIN-only accesses

User PIN › PIN-only access › People with a PIN only (keepers and section managers). Enter a name, an e-mail address and a group (the group is optional). A technical user without a usable password stands behind it.

11.1 Signing in

- The person gets a personal link by e-mail (valid 7 days for the first setup), chooses a PIN of their own (8+ characters) and signs in with the link and the PIN.
- From then on everybody uses **one shared page** (`/pin/` by default; a keeper chooses the address on the PIN-only page, e.g. `/tenants/`; case-insensitive; to be saved as a bookmark) and enters **only the PIN**: the PIN tells who the person is, so two people never hold the same PIN (the second is asked to choose another).
- An address list (IPs / networks) per access restricts the computers.
- E-mails to PIN-only people go to their own address whatever the allowed domains are.

11.2 Limits of the person

- No password, no 2FA, no API, no token; never a keeper, a manager or an exception. The account pages and the pages of the plugin are closed to them (only *My access*, *Change my PIN*, *Sign out*).
- Not a NetBox user: hidden from every user list, drop-down and the API. The NetBox change log shows the technical user (`PinUser0001` …).
- Without a group they can only change their PIN. The drop-downs of forms work for them (brief API lists), the rest of the API stays closed. They save their own table columns.

11.3 Wrong PINs (default 5 per step, waits 10, 20, 30 s; own rules on the PIN-only page, also per access)

- On the **personal link** the person is known: after every 5 wrong PINs a wait, after the last step the access is **blocked** (the person and the keepers get an e-mail; only an administrator unblocks).

- On the **shared page** a wrong PIN cannot be put down to a person: the computer (its address) waits by the same steps, the last wait repeats, nobody is blocked, and the keepers get an e-mail after every full round (15).

11.4 Managing an access

- **Disable / enable:** the stop page says "disabled by X since …" with an "ask the administrator" button (at most once an hour).
- The **validity** of the access can be changed later (e-mails 30 days and 1 day before the end), a forced PIN change, an own language for the PIN pages and for NetBox, a stable ID `PinUser0001` with a free alias and description.
- **Session:** idle 30 min (5–240) and 8 h at most (1–24), settable per access, an optional list of IP addresses and an optional end.
- "I forgot my PIN" e-mails a link for a new one (1 hour, once); not for a disabled or ended access.
- A keeper can send a new link (the old one stops), clear the PIN, revoke (ends every session), restore and delete the access.

11.5 E-mails both ways

- The **person** gets an e-mail about every change an administrator makes (disable, enable, PIN reset, validity, languages, rules, working time). The switch *E-mail the person about changes of the access* is global and per access.
- The **administrators** get a summary of what the person changed in NetBox, either **every N minutes** (60 by default) or **at every change** (never both). Subject: *PinUser0001 made a change*; body: time, object and the changed fields (from → to). Sent after the save through `netbox-rq`, so saving stays fast. The switch *E-mail the administrators what the person changed* is global and per access.

12. Working time

The general rules are on the PIN-only page: **country** (for the public holidays), **working days** (Mon–Fri by default) and **hours** (06:00–16:00 by default); the switch is *Limit the working time of the PIN-only accesses* (off by default). Every access may have its own rules (e.g. a Hungarian technician with the Hungarian holidays), the general ones, or **none**.

- Public holidays come from the `holidays` library (works offline, needs `pip install holidays`; without it only days and hours are checked). Countries: SK, CZ, HU, AT, PL, DE, UA, HR, SI, RO, BG, RS, GB, IE, FR, ES, IT, NL, BE, CH, SE, NO, DK, FI, PT, GR, LT, LV, EE, US, CA.
- The holidays follow the law the library implements. For example in Slovakia in 2026 the days off are 1 Jan, 6 Jan, Good Friday, Easter Monday, 1 May, 5 Jul, 29 Aug, 1 Nov and 24, 25, 26 Dec; 8 May, 1 Sep, 15 Sep and 17 Nov are working days. The plugin has no holidays of an employee and no custom company days.
- **Outside the working time** the person can sign in and read, but **cannot save or change anything** (no save, no add/edit/delete page, no API write). The working time is always shown (welcome page and top bar, in the language of the person).
- When the time ends while they work, they have **30 minutes and one save**, then only reading.

13. E-mails at a glance

Who gets it	When
User	PIN changed / reset / locked / expiring; invitation; recovery code; backup codes shown
CORE deputy, heads, delegates	invitations, resignations, hand-overs (a reminder the day before), moves
Approver	a new four-eyes request
Keepers	an exception, emergency commands, a full round of wrong PINs, a blocked access, requests, an access created by a manager
PIN-only person	every change by an administrator; the end of the validity (30 days and 1 day before)
Section managers	requests for their types and the decisions

14. Audit log

An append-only record of every PIN event (it never contains a PIN, hash or code). It shows settings, keys, roles, departments, approvals, areas (requests, grants, revocations, refused accesses, managers), keepers, switching the protection on/off, exceptions and their changes, and everything around the PIN-only accesses (creation, invitation, sign-in, wrong PINs, blocking, rules, manager, working time, expiry). Heads and delegates see their own department, CORE and the master everything.

15. Server commands

Command	What it does
`userpin_generate_key [--key-id k1]`	generates a random encryption key (prints it, stores nothing).
`userpin_rotate_key`	re-encrypts everything under the active key (PIN hashes, 2FA, backup codes, SMTP password).
`userpin_check`	verifies that everything stored under the keys can be decrypted and shows how many values and PIN-only accesses still wait for an old key.
`userpin_keeper [--add name]`	shows the keepers or appoints one (emergency, audited, e-mailed to everybody).
`userpin_default_deny [--on|--off]`	shows or switches the global protection (emergency, audited, e-mailed to the keepers).
`userpin_area_owners <area> [--set names…] [--force]`	shows or replaces the managers of an area (emergency, audited).
`userpin_reset_2fa <user>`	removes the 2FA in an emergency (e.g. the master lost the phone and all backup codes).
`userpin_access_test <user> [--url …] [--all] [--writes]`	shows what a user gets from the pages, the REST API and the lists (a test of the protection); `--all` tries every page, `--writes` also an add, a change and a delete.

All are run as `sudo bash -c 'source /opt/netbox/venv/bin/activate && cd /opt/netbox/netbox && python manage.py <command>'`.

15.1 Key rotation

1. Add the new key next to the old one and make it active: `'encryption_keys': {'k1': '<old>', 'k2': '<new>'}, 'active_key_id': 'k2'`.
2. Restart NetBox and run `userpin_rotate_key`.
3. Verify with `userpin_check`. The index of the PIN-only page moves to the new key only when each person signs in again: **remove the old key only when `userpin_check` says so** (it shows how many still wait).

The plugin never generates or switches keys by itself: a missing or wrong key is reported as an error. Store the new key aside (a password manager) and back up the database before the change.

16. Check: pin_check.sh

The script `tools/pin_check.sh` checks everything that can be checked from outside (and with `--server` on the server). It never stores or prints a PIN or a token.

```
bash pin_check.sh http://172.30.1.25                       # anonymous checks only
bash pin_check.sh http://172.30.1.25 --pin /pin/           # + signs in a PIN-only person (the PIN
is asked silently)
NB_TOKEN=xxxx bash pin_check.sh http://172.30.1.25         # + checks with an API token
sudo bash pin_check.sh http://127.0.0.1 --pin /pin/ --server   # + migrations, services, keys, keep
ers
```

Options: `--pin <path>` (address of the PIN-only page), `--wrong` (tries ONE wrong PIN; it counts as a wrong attempt!), `--write` (tries to add a tenant as the PIN-only person and deletes it), `--server` (needs root). Variables: `NB_TOKEN`, `NB_VENV`, `NB_DIR`, `LIST`. The result: the number of OK / FAIL / WARN. Known harmless warnings: `/static/docs/` is served by nginx (can be closed with `location /static/docs/ { return 404; }`) and `LOGIN_REQUIRED` is not `True` (set it in `configuration.py`).

17. Security model

```
PIN ──Argon2id + salt──► hash ──AES-256-GCM (key from configuration, bound to the user id)──► datab
ase
```

- A database dump alone contains nothing usable, not even something to brute-force offline.
- The ciphertext is bound to its user: a hash copied into another user's row does not decrypt.
- Wrong attempts are counted under a row lock, so parallel guessing cannot bypass the lockout.
- TOTP secrets are encrypted the same way; a TOTP code is accepted only once.
- Backup codes (100 bits) and e-mailed recovery codes are stored only as digests, single use, short-lived.
- Recovery never bypasses 2FA. The PIN is only an authentication factor ("it is really you at this computer"); it is not used as an encryption key for other data.
- Requests with an API token are not judged by the PIN (a token carries NetBox permissions): that is why tokens are off by default and are allowed one by one for exceptions.
- Access to the server is ruled by the firewall; the plugin does not enforce a second port or hostname for the PIN-only sign-in, this is solved in nginx and the firewall if needed.

18. Using it from another plugin

```
from netbox_user_pin.mixins import PinRequiredMixin, StepUpRequiredMixin, pin_required, step_up_req
uired

class ProjectView(PinRequiredMixin, generic.ObjectView):
    pin_scope = 'projects'          # used when the scope mode is "per scope"

class ProjectDeleteView(StepUpRequiredMixin, generic.ObjectDeleteView):
    ...                             # sensitive action: fresh PIN + 2FA

@pin_required(scope='projects')
def download(request, pk): ...
```

Service API `netbox_user_pin.service`: `has_pin(user)`, `is_unlocked(request, scope)`, `unlock(request, pin, scope)`, `verify_pin(user, pin)`, `lock(request)`, `has_step_up(request)`, `has_2fa(user)`, `is_allowed(user)`; `API_VERSION` is 2 and the API only grows. Signals: `pin_set`, `pin_changed`, `pin_reset`, `pin_unlocked`, `pin_failed`, `pin_locked_out`, `pin_locked`. An area of a plugin:

```
from netbox_user_pin import areas
areas.register('netbox_libreoxi', 'LibreOXI', targets=['netbox_libreoxi'],
               rights={'config': 'View device configurations'},
               profiles=[('Logs only', ['view']), ('Configurations', ['view', 'config'])],
               enabled=lambda: LibreOXISettings.objects.first().pin_integration)
areas.require(request, 'netbox_libreoxi', 'config')   # the "no access - request it" page
areas.can(request.user, 'netbox_libreoxi', 'config')  # True / False
```

19. Troubleshooting

Message / symptom	Cause and solution
`ModuleNotFoundError: No module named 'argon2'`	With `--no-deps` the dependencies were not installed. Run `pip install "argon2-cffi>=23.1" "cryptography>=42" "segno>=1.5" "holidays>=0.50"`.
`CommandError: No installed app with label 'netbox_user_pin'`	The plugin is not in the `PLUGINS` list in `configuration.py`. Add `'netbox_user_pin'` and repeat `migrate`.
pip: *holidays requires python-dateutil<3 but you have 3.0.0*	A harmless warning: `holidays` works with dateutil 3 (verify by listing the holidays). Do not install `python-dateutil<3`, you could break NetBox.
`userpin_check` reports an error	A wrong or missing key. Do not change the key `k1` in the configuration until the plugin is confirmed to work.
A user without rights sees Admin → API Tokens	NetBox standard (`DEFAULT_PERMISSIONS`). For an exception the plugin hides it unless "API tokens" is allowed.
403 on an account page for an exception	That part is not allowed: the keeper ticks it with the exception.
A PIN-only person can only read	It is outside the working time (or a holiday), or the protection is off. See the Working time chapter and the Protection card.
A PIN-only person sees nothing	No group, or the protection is off.
Adding a user to a group gives him nothing	The user has an exception: the group does not apply while the exception lasts.

netbox / netbox-rq do not start after an upgrade	Check `journalctl -u netbox -n 80`; most often a missing dependency or a plugin dropped from `local_requirements.txt`.
The upgrade put a plugin back to an older version	The pinned tag/commit is missing in `local_requirements.txt`, or the line has no `@…` (then `main` is taken).

20. Changes

The version in the code is **2.0.0** (the GitHub release) and **2.0.1** (branch `feature/exception-allowlist`). The original base was 0.38.4 (commit 7bed7de); the interim numbers 2.0.1 to 2.2.2 in the commit history were working labels, everything is included in the current 2.0.0 line and is split by topic below.

20.1 2.0.1 – permissions of exceptions

- An exception is no longer "all or nothing": the keeper ticks what the user may use (the password allowed by default; profile, preferences, bookmarks, notifications, subscriptions, API tokens and the API with a token to be allowed).
- What is not allowed is hidden from the menu and the tabs and refused by the address (403); a token without the allowed API is refused. A keeper is never limited.
- A "Change the password" link in the user menu; the bell is hidden when notifications are not allowed.
- An arrow ▾ under the name of the user with check boxes and green/red tags; an existing user is not offered again and adding one again resets nothing.
- New field `PolicyException.allowed` (migration `0042`); Slovak translation.

20.2 Keys and security

- Key rotation covers 2FA, backup codes, the SMTP password and the PIN-only index too; the command `userpin_check` verifies all encrypted values.
- Removing the PIN of a user without 2FA; `userpin_area_owners` checks eligibility.
- The master manages the PINs while there is no CORE deputy, even with existing keepers.

20.3 The new model of protection

- The PIN rules only the PIN-only accesses and the users in a group or section managers; everybody else works by the NetBox permissions (menu and API keys included).
- PIN-only accesses are no NetBox users (hidden from lists), have no requests and can only change the PIN without a group.
- Protection ON rules everybody but keepers and exceptions; OFF leaves NetBox alone and pauses the PIN-only accesses. The ON/OFF switch is in the interface (keepers only) and shows the real state. Exception vs. group: the group waits under the exception.
- Protection keepers: the last-keeper rule, emergency commands (`userpin_keeper`, `userpin_default_deny`).

20.4 Section managers and requests

- A section manager with full power over his section (the keepers above); PIN-only accesses under a manager; requests to the manager and the keepers with e-mails about the decisions.
- Appointed only by a keeper; relaxed eligibility (PIN + 2FA, no department role).

20.5 PIN-only accesses

- E-mails both ways: the person about the changes of an administrator; the administrators a summary every N minutes or at every change (not both), subject *PinUserNNNN made a change*, body: time, object, changed fields (from → to); sent after the save through `netbox-rq`.
- The drop-downs of forms work; saving own table columns.
- Conditions on relation fields: contains / starts with / is one of, several values, the Conditions button.

20.6 Working time (since 2.2)

- General and own rules, a country with its public holidays (the `holidays` library), working days and hours; the working time always shown.
- Read-only outside the time; 30 minutes and one save after the end; the country in the language of the person, days as Mon–Fri.

20.7 Tools and upgrade preparation

- `tools/pin_check.sh` – a check from outside and on the server; the dependency `holidays` in `pyproject.toml`.
- A prepared `local_requirements.txt` for upgrading NetBox within 4.7.x.
- Slovak translations, a README with the 2.x model, the version label 2.0.0.

20.8 What the plugin does not do yet

- Vacations and custom company days in the working time.
- The API for regular users in a group is still governed by the group rights (not fully closed).
- A separate port/hostname for the PIN-only sign-in (solved in nginx and the firewall).